

Pestaña 1

Propuesta de Proyecto DBP 2026-1

Integrantes:

- Kate Alva Torres - 201910062
- Micaela Garro - 202410675
- Héctor Choque - 202510780
- Israel Carlos - 202110052
- Royer Ramos - 202510069

Proyecto: Yala — Marketplace de subastas de coleccionables

1. El Problema y la Solución

¿Qué problema resuelve y para quién? Los coleccionistas del mundo geek de cartas Pokémon TCG, cómics y figuras Funko Pop en Latinoamérica no cuentan con una plataforma especializada y confiable para comprar, vender y subastar sus ítems. Los canales actuales (grupos de Facebook, Mercado Libre) carecen de sistema de grado/condición estandarizado, subastas en tiempo real y verificación de vendedores, lo que genera desconfianza y fraudes frecuentes.

Solución en una frase: Una aplicación web y móvil que conecta coleccionistas para subastar cartas Pokémon TCG y Funkos, con sistema de condición/grado estandarizado (PSA), subastas en tiempo real y perfiles de vendedor verificados.

2. Funcionalidades Principales

Las 7 cosas más importantes que un usuario podrá hacer:

1. Registrarse como comprador y/o vendedor, con perfil público y reputación
2. Publicar un coleccionable con fotos, categoría, condición (PSA/estado) y precio
3. Explorar y buscar ítems con filtros por tipo (Pokémon / Funko), precio y condición
4. Participar en subastas en tiempo real: pujar, ver historial de ofertas y recibir notificaciones al ser superado
5. Crear una subasta como vendedor, definiendo precio base y duración
6. Recibir notificaciones (email/push) al ganar una subasta, ser superado en puja o concretarse una venta

MVP — Funcionalidades imprescindibles (máximo 4):

- Registro y autenticación de usuarios
- Publicar listings (subasta)
- Sistema de subastas en tiempo real con temporizador
- Compra directa y confirmación de transacción

3. Estructura de Datos

3.1 Entidades del sistema:

#	Entidad	Atributos
1	USUARIO	id, nombre, email, contraseña_hash, avatar_url, reputacion, es_vendedor_verificado, fecha_registro
2	CATEGORIA	id, nombre, descripcion
3	PUBLICACION	id, titulo, descripcion, modalidad (fijo subasta), precio_fijo, condicion, estado, fecha_creacion, vendedor_id FK, categoria_id FK
4	IMAGEN	id, publicacion_id FK, url, orden
5	ETIQUETA	id, nombre
6	PUBLICACION_ETIQUETA	publicacion_id FK, etiqueta_id FK
7	SUBASTA	id, publicacion_id FK, precio_base, precio_actual, fecha_inicio, fecha_fin, estado, ganador_id FK
8	PUJA	id, subasta_id FK, pujador_id FK, monto, fecha_puja

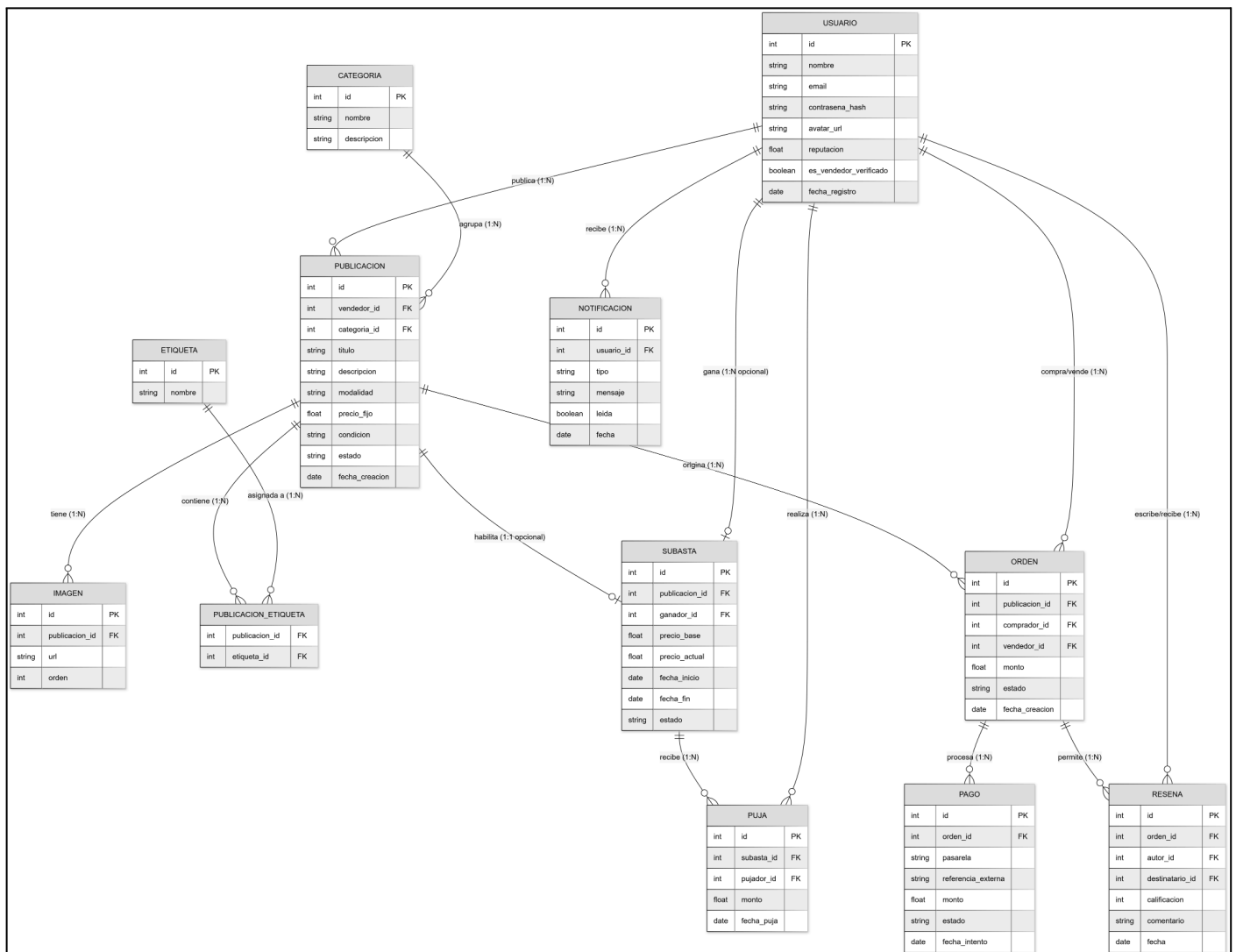
9	ORDEN	id, publicacion_id FK, comprador_id FK, vendedor_id FK, monto, estado (pendiente confirmada cancelada), fecha_creacion
10	PAGO	id, orden_id FK, pasarela (stripe mercadopago), referencia_externa, monto, estado (pendiente exitoso fallido reembolsado), fecha_intento
11	RESENA	id, orden_id FK, autor_id FK, destinatario_id FK, calificacion, comentario, fecha
12	NOTIFICACION	id, usuario_id FK, tipo, mensaje, leida, fecha

3.2. Relaciones

Relación	Cardinalidad	Notas
USUARIO → PUBLICACION	1:N	Un vendedor publica muchos ítems
CATEGORIA → PUBLICACION	1:N	Una categoría agrupa muchas publicaciones
PUBLICACION → IMAGEN	1:N	Múltiples fotos por publicación
PUBLICACION ↔ ETIQUETA	N:M	Via PUBLICACION_ETIQUETA
PUBLICACION → SUBASTA	1:1 opcional	Solo si modalidad es subasta
SUBASTA → PUJA	1:N	Múltiples pujas por subasta
USUARIO → PUJA	1:N	Un usuario puede pujar en varias subastas
USUARIO → SUBASTA (ganador)	1:N opcional	El usuario que gana la subasta
PUBLICACION → ORDEN	1:N	Una publicación puede originar varias órdenes

USUARIO → ORDEN (comprador)	1:N	Un usuario puede tener muchas órdenes
USUARIO → ORDEN (vendedor)	1:N	Un usuario puede vender en muchas órdenes
ORDEN → PAGO	1:N	Una orden puede tener varios intentos de pago
ORDEN → RESEÑA	1:N	Solo tras orden completada
USUARIO → RESEÑA (autor)	1:N	El usuario que escribe la reseña
USUARIO → RESEÑA (destinatario)	1:N	El usuario que recibe la reseña
USUARIO → NOTIFICACION	1:N	Un usuario recibe múltiples notificaciones

3.3 Diagrama de Entidad y Relaciones



4. Aspectos Técnicos

Servicios externos:

- **Stripe/Mercado pago** — procesamiento de pagos entre usuarios
- **Supabase** — almacenamiento y optimización de imágenes de los coleccionables
- **Firebase Cloud Messaging** — notificaciones push (puja superada, subasta ganada, venta confirmada)
- **Resend** - servicio de email
- **Socket.io (WebSockets)** — comunicación en tiempo real para las subastas (actualización de puja actual, temporizador, historial en vivo)

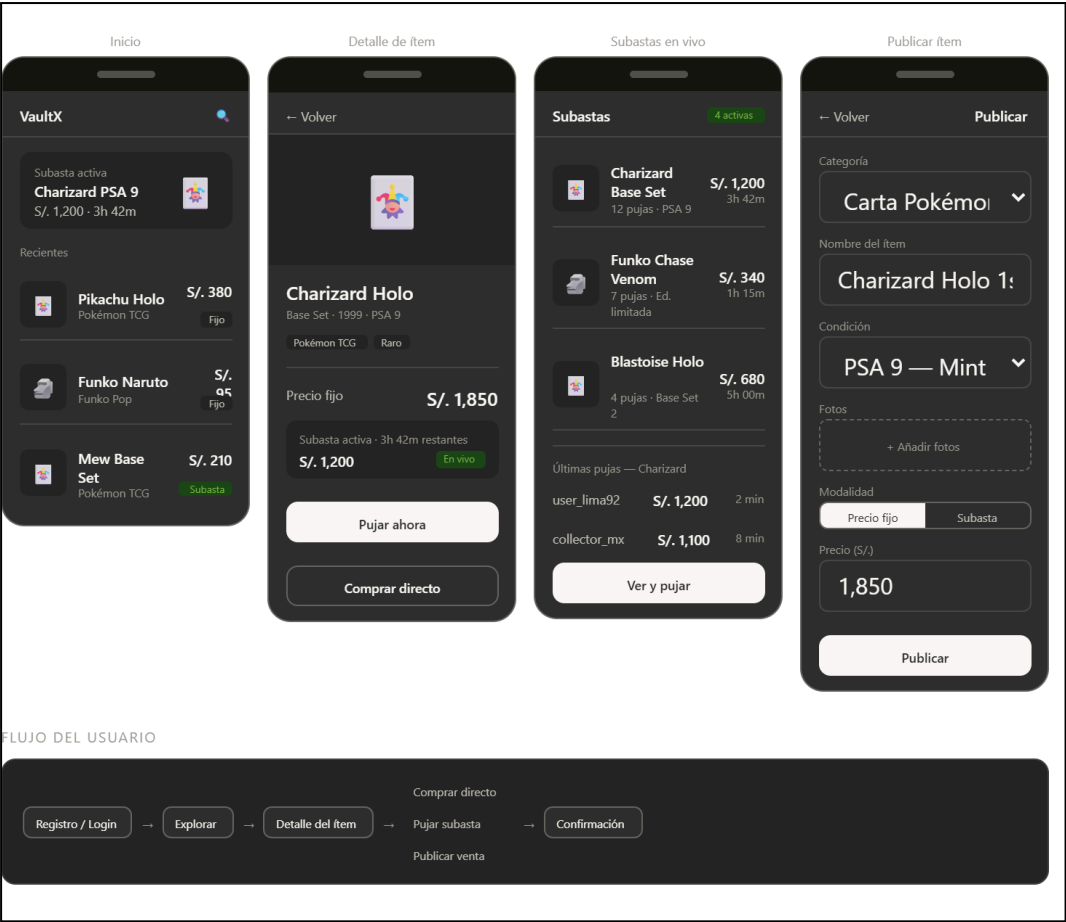
Operaciones que necesitan manejo asincrónico:

- Procesamiento de pagos con Stripe (llamada externa, latencia variable)
- Subida y procesamiento de imágenes a Supbase
- Envío de notificaciones push a múltiples usuarios simultáneamente (ej: cuando una subasta termina)
- Cierre automático de subastas al vencer el temporizador (tarea programada/cron job en el backend)
- Actualización en tiempo real del precio actual y contador de pujadores vía WebSockets

5. Mockups / Wireframes

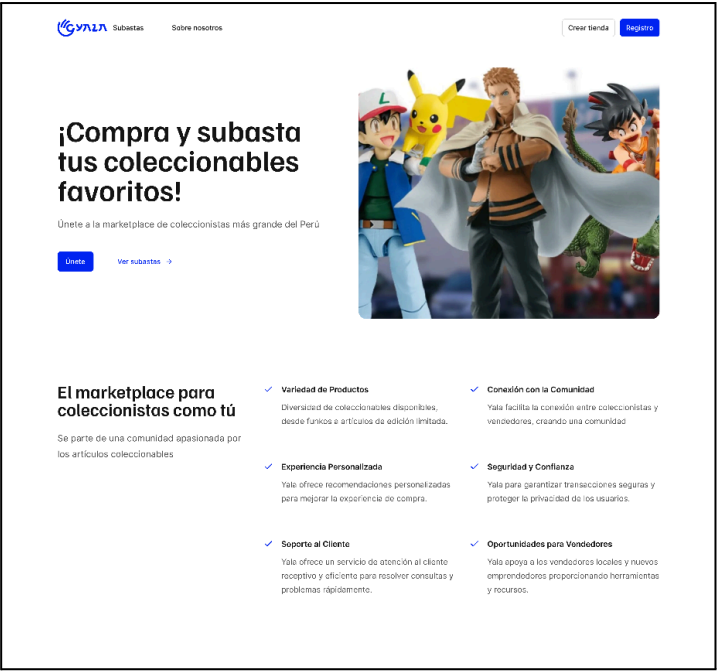
5.1 Propuesta 1

Inicio, Detalle de ítem, Subastas en vivo, Publicar ítem — más el diagrama de flujo de usuario)

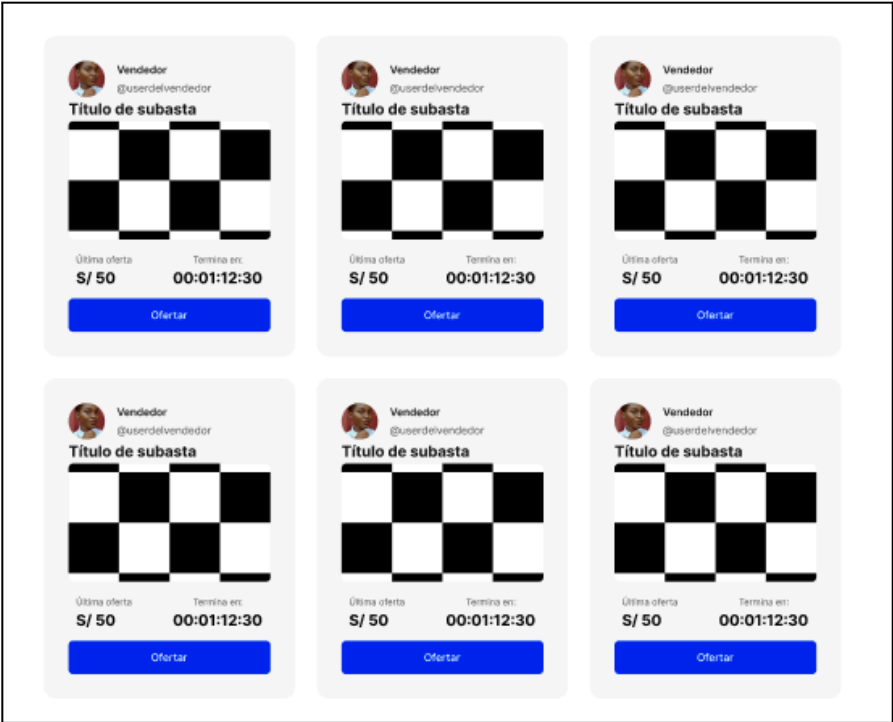


5.1 Propuesta 2

Pantalla principal informativa sobre Yala y botón de registro para usuarios o creación de tienda.

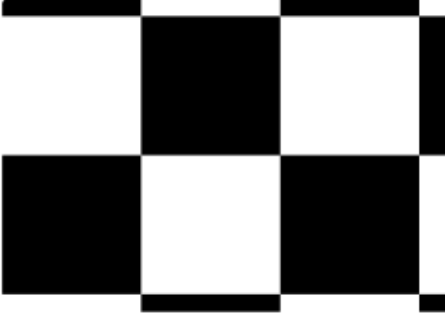


Subastas en línea



Detalle de la subasta en donde el usuario (fanático de los coleccionables del mundo geek) puede participar

Título de la subasta



S/. 55

Ofertar

 Envío a S/. 5 a toda lima

Lista de ofertas

Hora	Usuario	Monto
Dato	Dato	Dato
Dato	Dato	Dato
Dato	Dato	Dato
Oferta inicial	@vendedor	S/. 50.00

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Vestibulum ac accumsan purus, in interdum neque. Interdum et malesuada fames ac ante ipsum primis in faucibus. Quisque placerat dolor nisi, a condimentum ipsum sollicitudin id. Quisque in ante vel ex lacinia tempor id accumsan orci. Praesent at convallis elit. Quisque posuere luctus imperdiet. Morbi sodales libero scelerisque odio vestibulum, vitae eleifend lorem ornare. Aliquam erat volutpat. Fusce rutrum convallis eros, ut mollis nunc volutpat quis. Maecenas tristique est a cursus blandit. Maecenas non eros enim. Aenean lectus diam, cursus eu auctor sed, ornare vitae urna. Nullam at vestibulum diam. Vivamus ac maximus nisi, et pulvinar purus.



Registro de tienda (el cliente que tiene los productos)

¡Hora de tener tu tienda en Yala!

Completa los datos para solicitar la creación de tu tienda 🚀

← Regresar

Creación de tienda

Nombre de la tienda

Collector Store

Tipo de documento

☐ DNI

☒ RUC

☐ CE

Número de documento

20987654321

Dirección de la tienda

Jr. Santo Domingo 123, Barranco, Lima

Correo electrónico

correo@ejemplo.com

Contraseña

Enviar solicitud

Creación de la subasta por parte de la tienda

Mis subastas

+ Crear subasta

Mi cuenta

Dashboard

Mis datos

Configuración

Tienda X

Soporte

Salir

Editar subasta

Título de la subasta

Funko de Spiderman

Descripción

Funko especial de primera edición...

Categoría

Seleccionar

Estado

Seleccionar

Envío

☐ Envío a cargo del comprador

☒ Envío gratis

Imagen del producto 1

Elegir archivo

Imagen del producto 2

Elegir archivo

Imagen del producto 3

Elegir archivo

Guardar cambios

Registro de los usuarios (fanáticos del mundo geek) para poder participar en la subasta

¡Compra y subasta tus coleccionables favoritos!

Únete a la marketplace de coleccionistas más grande del Perú

[← Regresar](#)

Completar cuenta

DNI

Cumpleaños

Banco

Código de Cuenta Interbancario (Solo si publicarás subastas)

Dirección

Número de celular

Actualizar y validar

Omitir →

Link de Figma:

<https://www.figma.com/design/VHNeB6uk6hXL12nu6xb01/Yala-UI?node-id=9270-6134&t=FuD5xcrx3AEAbj5i-0>

- Pantalla principal (Home con feed, filtros y subasta destacada)
- Detalle de ítem (subasta activa + botones de acción)
- Subastas en vivo (listado con temporizador e historial de pujas)
- Publicar ítem (formulario con selección de modalidad: subasta)
- Flujo de usuario: Registro → Explorar → Detalle → Comprar/Pujar/Publicar → Pago/Confirmación

6. Reflexión Final

¿Qué parte será la más desafiante?

El mayor reto técnico será implementar el **sistema de subastas en tiempo real** con WebSockets: garantizar consistencia de datos cuando múltiples usuarios pujan simultáneamente (condiciones de carrera en la base de datos), manejar el cierre automático de subastas con precisión temporal mediante cron jobs, y sincronizar el estado en todos los clientes conectados sin pérdida de eventos. Adicionalmente, la integración de Stripe/Mercado pago para la paga de las subastas y manejo de errores de transacción representa un desafío de seguridad y lógica de negocio no trivial.

¿Experiencia previa del equipo?

Tenemos experiencia con APIs REST y bases de datos relacionales del curso, pero ninguno ha implementado comunicación en tiempo real con WebSockets ni integrado un gateway de pagos real como Stripe/Mercado Pago. Conocemos React para el frontend pero no hemos desplegado una aplicación completa en producción con estas características.

YALA



Marketplace de Subastas de Coleccionables

Documento de Logica de Negocio

CS2031 - Desarrollo Basado en Plataforma | UTEC 2026-1

Integrantes:

Kate Alva Torres - 201910062

Micaela Garro - 202410675

Hector Choque - 202510780

Israel Carlos - 202110052

Royer Ramos - 202510069

1. Vision General del Negocio

Yala es un marketplace especializado en subastas de articulos coleccionables del mundo geek (cartas Pokemon TCG, figuras Funko Pop y comics) dirigido al mercado latinoamericano. La plataforma conecta coleccionistas y tiendas especializadas mediante un sistema de subastas transparente, con condicion estandarizada PSA, perfiles verificados y comisiones claras.

1.1 Actores del Sistema

Actor	Rol en el Sistema	Descripcion
Usuario Comprador	BUYER	Puede explorar y ver el marketplace sin restricciones. Para pujar debe completar la verificacion de identidad ingresando su DNI (8 digitos numericos).
Usuario Pujador Verificado	SELLER (usuario)	Usuario que ha participado en al menos 5 subastas anteriores en la plataforma, de las cuales ha ganado un minimo de 3, y ha solicitado verificacion para publicar sus propias subastas.
Tienda	SELLER (tienda)	Entidad comercial registrada que puede publicar articulos en subasta desde el inicio, sin requisito previo de participacion.
Administrador	ADMIN	Gestiona categorias, aprueba verificaciones de usuarios, modera publicaciones, revisa disputas y tiene acceso total al sistema.

2. Registro y Autenticacion

2.1 Registro de Usuario Comprador

Flujo basico para cualquier persona que desee explorar y comprar en la plataforma:

- El usuario ingresa: nombres y apellidos, email, contrasena (minimo 8 caracteres, al menos 1 mayuscula y 1 numero) y fecha de nacimiento. El DNI es opcional en este paso — puede completarlo despues. Debe ser mayor de 18 anos.
- El sistema valida que el email no este registrado previamente. Si ya existe, retorna error 409 Conflict.
- Se hashea la contrasena con BCrypt antes de almacenarla. Nunca se guarda en texto plano. La cuenta se crea con estado DNI_PENDIENTE hasta que complete la verificacion de identidad.
- Se dispara el evento UsuarioRegistradoEvent de forma asincronica para enviar un email de bienvenida.
- El sistema retorna un JWT con rol USER y el perfil basico del usuario. Con este estado puede navegar y explorar el marketplace libremente, pero no puede pujar ni comprar hasta completar la verificacion de identidad.

2.2 Verificacion de Identidad

Cuando el usuario intenta pujar o comprar por primera vez, el sistema le bloquea el boton de accion y muestra un modal solicitando completar su verificacion de identidad. El flujo es el siguiente:

- El usuario ingresa su numero de DNI (exactamente 8 digitos numericos), sus nombres y sus apellidos. Todos los campos son obligatorios.
- El sistema valida que: el DNI tenga exactamente 8 digitos numericos (@Pattern), los nombres no esten vacios ni contengan numeros o caracteres especiales (@NotBlank + @Pattern con solo letras y espacios), y los apellidos cumplan las mismas reglas. Si alguna validacion falla, retorna 400 Bad Request con el detalle del campo invalido.
- Si todas las validaciones pasan y el DNI no esta duplicado, el sistema registra el DNI, nombres y apellidos en la cuenta del usuario y actualiza su estado a dniVerificado = true. El usuario puede proceder a pujar inmediatamente sin recargar la pagina.
- El DNI, nombres y apellidos quedan almacenados en la base de datos asociados unicamente a esa cuenta.
- Si el DNI no tiene exactamente 8 digitos, o los nombres y apellidos contienen caracteres invalidos o estan vacios, el sistema muestra un mensaje de error descriptivo por campo y el usuario puede corregirlos e intentarlo de nuevo.
- Un mismo DNI solo puede estar asociado a una cuenta activa en la plataforma. Si el DNI ya esta registrado en otra cuenta, se retorna error 409 con mensaje: 'Este DNI ya esta asociado a otra cuenta'.
- El DNI se almacena en la base de datos. En el perfil publico del usuario no se muestra el DNI, solo aparece el icono de 'Identidad Verificada'.

Mientras el usuario navega sin verificar su identidad puede: ver publicaciones, ver subastas activas, ver historial de pujas, y ver perfiles publicos. No puede: pujar ni crear publicaciones.

2.3 Registro de Tienda

Las tiendas tienen un flujo de registro mas completo dado que pueden operar como vendedores desde el dia uno:

- La tienda proporciona: nombre de tienda, direccion, email, contrasena, codigo de cuenta interbancaria (CCI) y numero de celular.
- El CCI es obligatorio porque es la cuenta donde Yala deposita el monto neto de cada subasta ganada (precio final menos la comision del 8%).
- El sistema valida unicidad de email.
- La tienda se crea con estado PENDIENTE_VERIFICACION. El ADMIN revisa los datos y activa la cuenta.
- Una vez aprobada, la tienda recibe rol SELLER y puede publicar articulos y crear subastas sin restricciones.
- Se envian emails de confirmacion al momento de solicitud y al momento de aprobacion.

2.4 Verificacion de Usuario para Publicar Subastas

Un usuario comprador puede solicitar ser verificado como vendedor si cumple el requisito minimo:

- Requisito: haber participado en al menos 5 subastas en la plataforma Y haber ganado un minimo de 3 de ellas. Ambas condiciones deben cumplirse simultaneamente.
- El usuario envia una solicitud de verificacion desde su perfil. El sistema verifica automaticamente el conteo de subastas en las que participo.
- Si no cumple el requisito, el sistema rechaza la solicitud con mensaje descriptivo indicando cuantas subastas le faltan.
- Si cumple, la solicitud queda en estado PENDIENTE para revision del ADMIN.
- El ADMIN puede aprobar o rechazar. Si aprueba, el usuario obtiene rol SELLER y puede publicar subastas.
- El usuario debe proporcionar su CCI antes de poder recibir pagos como vendedor.

2.5 Login y Sesion

- Login mediante email + contrasena. El sistema retorna un JWT con duracion de 24 horas.
- El JWT incluye: userId, email, rol (USER / SELLER / ADMIN) y fecha de expiracion.
- Todos los endpoints protegidos requieren el JWT en el header Authorization: Bearer {token}.
- Si el token expira, el usuario debe volver a hacer login.

3. Gestion de Publicaciones

3.1 Tipos de Publicacion

Yala es un marketplace de subastas de coleccionables donde cada articulo publicado pasa por un proceso de subasta, garantizando transparencia en la formacion del precio y competencia justa entre compradores.

Quien puede crear: Solo usuarios con rol SELLER (tienda registrada o usuario verificado).

Como se vende: El articulo va al mejor pujador al cierre de la subasta.

Comision de Yala: 8% sobre el precio final de venta, descontado del monto que recibe el vendedor.

Tiempo limite: Definido por el vendedor al crear la subasta: 1, 3, 5 o 7 dias.

3.2 Datos de una Publicacion

- Titulo descriptivo (obligatorio, minimo 10 caracteres).
- Descripcion detallada del articulo (obligatorio).
- Categoria: Carta Pokemon TCG / Funko Pop / Comic / Otro.
- Condicion / Grado PSA: PSA 10 (Gem Mint), PSA 9 (Mint), PSA 8 (Near Mint), PSA 7 o inferior, Sin graduar (estado libre: Excelente, Bueno, Regular).

- Modalidad: Subasta. El vendedor define el precio base de apertura y la duracion.
- Precio: si es fijo, precio de venta; si es subasta, precio base minimo de apertura.
- Fotos: minimo 1, maximo 5 imagenes del articulo.
- Etiquetas opcionales: 'Primera Edicion', 'Holografico', 'Limitado', etc.

3.3 Estados de una Publicacion

Estado	Descripcion
BORRADOR	Creada pero no publicada. Solo visible para el vendedor.
ACTIVA	Visible para todos. Puede recibir compradores o pujadores.
VENDIDA	La orden fue confirmada y el pago procesado exitosamente.
CANCELADA	El vendedor la retiro o la subasta termino sin ganador.
SUSPENDIDA	El ADMIN la suspendio por incumplimiento de politicas.

4. Sistema de Subastas

4.1 Creacion de una Subasta

- Solo usuarios con rol SELLER pueden crear subastas.
- El vendedor define: precio base (puja minima de apertura), duracion (1, 3, 5 o 7 dias) y fecha/hora de inicio (inmediata o programada).
- El sistema calcula y almacena la fecha_fin automaticamente.
- La subasta inicia en estado PROGRAMADA si tiene fecha futura, o ACTIVA si es inmediata.
- Un cron job ejecuta cada minuto para revisar subastas PROGRAMADAS cuya fecha_inicio ya llego y las activa automaticamente.

4.2 Flujo de Pujas

- Solo usuarios autenticados pueden pujar (compradores y vendedores, pero no el propio vendedor de la subasta).
- La puja debe ser estrictamente mayor al precio_actual de la subasta.
- Se recomienda un incremento minimo del 1% sobre el precio actual para evitar pujas de centavos.
- Al registrar una puja exitosa, el sistema actualiza el precio_actual de la subasta.
- Se dispara PujaRealizadaEvent de forma asincronica: todos los usuarios que esten viendo la subasta en ese momento (participantes activos y espectadores) ven el precio actualizado en tiempo real dentro de la pagina de la subasta.

Adicionalmente, el usuario que tenia la puja mas alta anteriormente recibe una notificacion dentro de la aplicacion indicando que fue superado.

- El historial completo de pujas queda registrado con usuario, monto y timestamp.

4.3 Cierre Automatico de Subastas

El cron job de cierre se ejecuta cada minuto y procesa todas las subastas cuya fecha_fin ya vencio y siguen en estado ACTIVA:

1. El sistema busca la puja de mayor monto para esa subasta.
2. Si existe al menos una puja: la subasta pasa a CERRADA, se registra el ganador, se crea una Orden en estado PENDIENTE entre el ganador y el vendedor, y se dispara SubastaGanadaEvent.
3. Si no hay pujas: la subasta pasa a CANCELADA, se dispara SubastaSinPujasEvent de forma inmediata y la publicacion vuelve a estado ACTIVA para que el vendedor decida si repubblica una nueva subasta.
4. El evento SubastaGanadaEvent dispara email al ganador indicando que gano, el monto final y el plazo de pago de 48 horas. Al vendedor se le notifica dentro de la aplicacion indicando el monto que recibira descontando el 8%.

4.4 Estados de una Subasta

Estado	Descripcion
PROGRAMADA	Creada pero su fecha_inicio aun no ha llegado.
ACTIVA	En curso. Acepta pujas hasta fecha_fin.
CERRADA	Termino con ganador. Esperando pago.
CANCELADA	Termino sin pujas o fue cancelada por el vendedor antes de la primera puja.
PAGADA	El ganador pago y la orden fue confirmada.

4.5 Cancelacion de Subasta por el Vendedor

- El vendedor solo puede cancelar una subasta si aun no tiene ninguna puja registrada.
- Una vez que existe al menos una puja, la subasta no puede cancelarse para proteger a los pujadores.
- Si el vendedor cancela antes de la primera puja, la publicacion vuelve a estado BORRADOR.

5. Sistema de Pagos y Comisiones

5.1 Modelo de Comision

La comision del 8% aplica a todas las ventas realizadas en la plataforma. El monto se descuenta automaticamente del precio final que recibe el vendedor.

Concepto	Subasta
Precio final	S/. 1,000 (puja ganadora)
Comision Yala (8%)	S/. 80
Monto neto al vendedor	S/. 920
Quien paga la comision	El vendedor (se descuenta del monto final recibido)

5.2 Flujo de Pago del Ganador de Subasta

Cuando una subasta cierra con ganador, se inicia el siguiente flujo:

- Se crea una Orden en estado PENDIENTE con el monto de la puja ganadora.
- El ganador recibe un email indicando que gano la subasta, el monto a pagar y el plazo maximo de 48 horas. El pago se realiza directamente dentro de la pagina web de Yala.
- El ganador accede a la plataforma y completa el pago mediante Stripe (modo sandbox para el proyecto).
- El sistema registra el intento de pago en la entidad Pago con estado PENDIENTE.
- Si el pago es exitoso (Stripe confirma): la Orden pasa a CONFIRMADA, la Subasta pasa a PAGADA, la Publicacion pasa a VENDIDA, y se dispara OrdenConfirmadaEvent.
- El evento OrdenConfirmadaEvent envia un email al comprador confirmando que su pago fue procesado exitosamente. Al vendedor se le notifica dentro de la aplicacion que el pago fue recibido y que Yala procesara la transferencia del monto neto (precio final - 8%) a su CCI en un plazo de 1-3 dias habiles.
- Se registra en el sistema el monto_comision y el monto_netto_vendedor para trazabilidad.

5.3 Que Pasa si el Ganador No Paga en 48 Horas

- Un cron job diario revisa ordenes PENDIENTE cuya fecha de creacion supere las 48 horas.
- La Orden del ganador original pasa a CANCELADA con motivo PAGO_VENCIDO. El sistema identifica automaticamente al segundo mejor pujador (el que tenia el segundo monto mas alto en la subasta).
- Si existe un segundo mejor pujador, se le notifica dentro de la aplicacion: 'El ganador anterior no completo el pago. Tienes 48 horas para aceptar el item al precio de tu ultima puja'. Se crea una nueva Orden PENDIENTE para el segundo pujador.
- Si el segundo pujador no paga tampoco en 48 horas, o si no existia segundo pujador, la publicacion vuelve a ACTIVA para que el vendedor la republique o cree una nueva subasta.
- El usuario que no pago recibe una penalizacion leve en su perfil (contador de pagos_fallidos) visible para los vendedores.

- El vendedor recibe notificaciones dentro de la aplicacion en cada etapa: cuando el ganador no pago, cuando se ofrecio el item al segundo pujador, y cuando el item quedo disponible nuevamente si nadie mas acepto.

5.4 Manejo de Fallos de Pago

- Si Stripe retorna error en el intento de pago, el registro de Pago queda en estado FALLIDO y se envia un email al comprador notificando que su pago fue rechazado para que lo reintente.
- El usuario puede reintentar el pago (la Orden sigue PENDIENTE) hasta que se venza el plazo.
- Se guardan todos los intentos de pago para auditoria.
- Si el pago queda en estado inconsistente, el ADMIN puede intervenir manualmente.

6. Gestion de Ordenes

6.1 Estados de una Orden

Estado	Descripcion
PENDIENTE	Creada, esperando que el comprador complete el pago.
CONFIRMADA	Pago exitoso. El vendedor debe coordinar el envio.
CANCELADA	Pago vencido, fallo o cancelacion por el comprador (solo antes del pago).
EN_ENVIO	El vendedor marco el articulo como enviado.
COMPLETADA	El comprador confirmo la recepcion. Habilita la resena.
DISPUTADA	El comprador reporto un problema. El ADMIN interviene.

6.2 Reglas de Negocio de Ordenes

- Una orden solo puede ser resentida cuando esta en estado COMPLETADA.
- El vendedor marca la orden EN_ENVIO y puede ingresar un numero de seguimiento opcional.
- El comprador confirma la recepcion, lo que mueve la orden a COMPLETADA y habilita la seccion de resenas.
- Si el comprador no confirma recepcion en 15 dias desde EN_ENVIO, el sistema la completa automaticamente.

7. Sistema de Resenas

7.1 Reglas de Resenas

- Las resenas son mutuas: el comprador puede resenar al vendedor Y el vendedor puede resenar al comprador.
- Solo se puede resenar cuando la orden esta en estado COMPLETADA.
- Cada usuario puede dejar maxima 1 resena por orden. No se pueden editar ni eliminar una vez publicadas.
- La resena incluye: calificacion de 1 a 5 estrellas y comentario textual opcional.
- El promedio de calificaciones recibidas como vendedor se muestra en el perfil publico y afecta la reputacion.
- El promedio de calificaciones recibidas como comprador es visible para los vendedores antes de aceptar una puja.

7.2 Impacto en la Reputacion

- La reputacion del usuario se calcula como el promedio ponderado de todas sus resenas recibidas.
- Un usuario con reputacion menor a 2.0 puede ser suspendido preventivamente y revisado por el ADMIN.
- Los perfiles publicos muestran: numero total de transacciones, porcentaje de calificaciones positivas (4-5 estrellas) y comentarios recientes.

8. Notificaciones y Eventos Asincronicos

8.1 Eventos del Sistema

Evento	Disparado por	Acciones Asincronicas
UsuarioRegistradoEvent	AuthService al registrar	Email de bienvenida al registrarse en la plataforma
PujaRealizadaEvent	PujaService al crear puja	Actualizacion en tiempo real del precio y historial para todos los que esten viendo la subasta (participantes y espectadores). Notificacion in-app al pujador anterior indicando que fue superado.
SubastaGanadaEvent	Scheduler al cerrar subasta	Email al ganador indicando que gano y plazo de pago de 48 horas. Notificacion in-app al vendedor con el monto neto que recibira.
SubastaSinPujasEvent	Scheduler al cerrar sin pujas	Notificacion in-app al vendedor de forma inmediata indicando que la subasta cerro sin participantes.
OrdenConfirmadaEvent	PagoService al confirmar pago	Email al comprador confirmando que el pago fue exitoso. Notificacion in-app al

Evento	Disparado por	Acciones Asincronicas
		vendedor indicando que el pago fue recibido.
PagoVencidoEvent	Scheduler al vencer 48hs	Email al comprador notificando que su pago fue rechazado o vencio. Notificacion in-app al segundo pujador ofreciendole el item. Notificacion in-app al vendedor en cada etapa.
TiendaAprobadaEvent	AdminService al aprobar tienda	Email de bienvenida a la tienda al momento de aprobacion con instrucciones de uso.
VerificacionAprobadaEvent	AdminService al verificar usuario	Notificacion in-app al usuario indicando que fue aprobado como vendedor.

8.2 Notificaciones In-App y Criterio de Uso

- Todas las notificaciones se almacenan en la BD con estado leida/no leida. El criterio de uso es el siguiente — Emails (solo 4 casos): registro del usuario o tienda en la plataforma (bienvenida), cuando el usuario gana una subasta (con monto y plazo de pago), cuando el pago fue procesado exitosamente, y cuando el pago fue rechazado o vencio. Tiempo real dentro de la pagina de la subasta: cuando se registra una nueva puja, todos los usuarios que esten viendo esa subasta en ese momento (participantes activos y espectadores) ven el precio y el historial actualizados instantaneamente sin recargar la pagina. Notificaciones in-app para todo lo demas: vendedor notificado del cierre sin pujas, vendedor notificado cuando el comprador pago, segundo pujador ofrecido el item, estado de envio y recepcion, aprobacion de verificacion como vendedor.
- El usuario puede ver su bandeja de notificaciones ordenada por fecha descendente.
- Puede marcar notificaciones como leidas individualmente o todas a la vez.
- El icono de notificaciones muestra el contador de no leidas en tiempo real.

9. Reglas de Negocio Criticas y Validaciones

9.1 Validaciones de Puja

Regla	Detalle
No pujar en propia subasta	El usuario que creo la subasta no puede pujar en ella. Error 403.
Monto minimo	La puja debe superar el precio_actual. Si hay empate exacto, se rechaza. Error 400.
Subasta activa	Solo se puede pujar si la subasta esta en estado ACTIVA. Error 400 con mensaje descriptivo.
Usuario autenticado	Sin JWT valido no se puede pujar. Error 401.

Regla	Detalle
Sin pujas duplicadas inmediatas	El mismo usuario no puede pujar dos veces seguidas sin que alguien mas pueje en medio.

9.2 Validaciones de Publicacion

- Un SELLER no puede tener mas de 20 publicaciones activas simultaneamente (limite para evitar spam).
- No se puede editar una publicacion que tiene ordenes pendientes o confirmadas asociadas.
- No se puede editar el precio_base de una subasta una vez que tiene al menos una puja.
- Las imagenes deben ser menores a 5MB cada una y en formato JPG, PNG o WEBP.

9.3 Concurrencia en Subastas

- Para evitar condiciones de carrera cuando multiples usuarios pujan simultaneamente, el sistema usa bloqueo optimista (@Version en la entidad Subasta).
- Si dos pujas llegan al mismo tiempo, la primera en procesarse gana y la segunda recibe un error indicando que el precio ya cambio, debiendola reintentar con el nuevo precio.

9.4 Seguridad de Datos

- Las contraseñas se hashean con BCrypt (strength 12) antes de almacenarse.
- El CCI del vendedor se almacena encriptado en la BD.
- Los datos de pago (numero de tarjeta, CVV) nunca pasan por los servidores de Yala; Stripe los maneja directamente via su SDK.
- Solo el propio usuario y el ADMIN pueden ver el CCI de una cuenta.
- Los endpoints sensibles (aprobar tiendas, suspender usuarios) estan protegidos con @PreAuthorize('hasRole(ADMIN)').

10. Resumen de Endpoints Principales

Metodo	Endpoint	Rol Requerido	Descripcion
POST	/api/v1/auth/register	Publico	Registro de usuario
POST	/api/v1/auth/login	Publico	Login, retorna JWT
POST	/api/v1/auth/register-tienda	Publico	Solicitud de registro de tienda

Metodo	Endpoint	Rol Requerido	Descripcion
GET	/api/v1/publicaciones	Publico	Listar con filtros y paginacion
GET	/api/v1/publicaciones/{id}	Publico	Detalle de publicacion
POST	/api/v1/publicaciones	SELLER	Crear publicacion
PUT	/api/v1/publicaciones/{id}	SELLER	Editar publicacion propia
GET	/api/v1/subastas	Publico	Listar subastas activas
POST	/api/v1/subastas/{id}/pujas	USER/SELLER	Realizar una puja
GET	/api/v1/subastas/{id}/pujas	Publico	Historial de pujas
POST	/api/v1/ordenes	USER/SELLER	Crear orden (subasta ganada)
POST	/api/v1/ordenes/{id}/pagar	USER/SELLER	Procesar pago de orden
POST	/api/v1/resenas	USER/SELLER	Crear resena (orden completada)
GET	/api/v1/notificaciones	USER/SELLER	Listar mis notificaciones
POST	/api/v1/admin/tiendas/{id}/aprobar	ADMIN	Aprobar registro de tienda
POST	/api/v1/admin/verificaciones/{id}/aprobar	ADMIN	Aprobar verificacion de usuario

11. Flujos de Usuario Completos

11.1 Flujo: Usuario gana una subasta y paga

- Usuario hace login y navega a 'Subastas en vivo'.
- Encuentra una subasta de Charizard PSA 9, precio actual S/. 1,200.
- Ingresa S/. 1,300 como puja. El sistema valida y registra la puja.
- El usuario anterior con S/. 1,200 recibe una notificacion dentro de la aplicacion: 'Tu puja fue superada'.
- Nadie mas puja. La subasta cierra automaticamente a las 3:00 AM del dia de vencimiento.
- El sistema detecta al ganador, crea una Orden PENDIENTE por S/. 1,300 y envia email al ganador.
- El ganador recibe: 'Ganaste la subasta. Tienes 48 horas para pagar S/. 1,300'.
- El vendedor recibe una notificacion dentro de la aplicacion: 'Tu subasta cerro. El comprador pagara S/. 1,300. Recibiras S/. 1,196 (92%)'.
- El ganador completa el pago dentro de la pagina web via Stripe. La orden pasa a CONFIRMADA. El ganador recibe un email confirmando que su pago fue procesado exitosamente.
- El vendedor recibe una notificacion dentro de la aplicacion confirmando el pago y coordina el envio.
- El vendedor marca la orden EN_ENVIO con numero de seguimiento.

- El comprador confirma recepcion. La orden pasa a COMPLETADA.
- Ambos pueden dejarse resenas mutuas.

11.2 Flujo: Tienda crea una subasta

- La tienda completa su registro con nombre, direccion, CCI y datos de contacto.
- El ADMIN aprueba el registro. La tienda recibe email de bienvenida.
- La tienda hace login y va a 'Crear Subasta'.
- Completa: 'Funko Pop Venom Ed. Especial', categoria Funko Pop, condicion Excelente, precio base S/. 150, duracion 3 dias.
- Sube 3 fotos del articulo a Supabase (el backend recibe las URLs).
- La subasta se activa y aparece en el feed publico.
- Los usuarios empiezan a pujar. Al cierre, el flujo de pago sigue el mismo camino del punto 11.1.

11.3 Flujo: Usuario solicita verificacion como vendedor

- Usuario tiene cuenta hace 2 meses y participo en 5 subastas (3 ganadas y 2 donde puso pujas pero no gano).
- Va a su perfil y hace clic en 'Quiero vender'.
- El sistema cuenta: 5 subastas participadas y 3 ganadas. Ambos requisitos cumplidos.
- El usuario completa su CCI y envia la solicitud.
- El ADMIN revisa y aprueba. El usuario obtiene rol SELLER.
- El usuario recibe email: 'Ya puedes crear subastas en Yala'.

12. Manejo de Errores y Excepciones

Excepcion	HTTP Status	Cuando ocurre
ResourceNotFoundException	404 Not Found	Subasta, publicacion, usuario o orden no encontrada por ID.
DuplicateResourceException	409 Conflict	Email ya registrado, resena duplicada en la misma orden.
InvalidBidException	400 Bad Request	Puja menor o igual al precio actual, puja en subasta propia.
AuctionClosedException	400 Bad Request	Intentar pujar en subasta que ya cerro o cancelo.
InvalidOperationException	400 Bad Request	Editar subasta con pujas, cancelar subasta con pujas.
UnauthorizedException	401 Unauthorized	Token JWT ausente, invalido o expirado.

Excepcion	HTTP Status	Cuando ocurre
ForbiddenException	403 Forbidden	Intentar operar recursos de otro usuario o sin el rol requerido.
PaymentException	500 Internal Server Error	Error inesperado al procesar el pago con Stripe.
VerificationRequiredException	403 Forbidden	Usuario sin verificacion intenta crear una subasta.
IdentityNotVerifiedException	403 Forbidden	Usuario intenta pujar sin haber registrado su DNI en la plataforma.

Todas las excepciones retornan un ErrorResponse con el siguiente formato:

Campo	Descripcion
timestamp	Fecha y hora exacta del error en formato ISO 8601.
status	Codigo HTTP numerico (400, 401, 403, 404, 409, 500).
error	Nombre corto del error (Bad Request, Not Found, etc.).
message	Descripcion legible y util del problema para el cliente.
path	Endpoint que genero el error (/api/v1/subastas/5/pujas).

13. Buenas Practicas de Desarrollo

Esta seccion describe los patrones, convenciones y decisiones tecnicas que el equipo debe aplicar de forma consistente en todo el codigo del proyecto Yala para garantizar calidad, mantenibilidad y alineacion con la rubrica del curso.

13.1 DTOs (Data Transfer Objects)

Nunca exponer una entidad JPA directamente en un endpoint. Siempre usar DTOs especializados segun la operacion:

- XxxRequest — datos que llegan del cliente (POST/PUT). Incluyen anotaciones de validacion (@NotNull, @Email, @Size, @Min).
- XxxResponse — datos que se retornan al cliente. Solo los campos necesarios, nunca contraseñas ni datos sensibles como el DNI completo o el CCI.
- XxxDetailResponse — version extendida del response para vistas de detalle (ej: SubastaDetailResponse incluye la lista de pujas, el vendedor y la publicacion completa).
- El mapeo entidad → DTO se centraliza en metodos estaticos dentro del propio DTO (metodo from(Entidad e)) o usando ModelMapper configurado como bean. No dispersar la logica de mapeo en los servicios.
- Los DTOs usan Lombok (@Getter, @Setter, @Builder, @NoArgsConstructor, @AllArgsConstructor) para evitar codigo boilerplate. Los records de Java 21+ son validos para DTOs inmutables de respuesta.

- ErrorResponse es un DTO especial retornado por el GlobalExceptionHandler. Siempre incluye: timestamp, status, error, message y path.

13.2 Manejo Global de Excepciones (@RestControllerAdvice)

- Existe una unica clase GlobalExceptionHandler anotada con @RestControllerAdvice. Ningun controller ni servicio atrapa excepciones con try/catch para construir respuestas HTTP — eso es responsabilidad exclusiva del handler global.
- Cada excepcion personalizada del proyecto (ResourceNotFoundException, InvalidBidException, etc.) tiene su propio metodo @ExceptionHandler en el GlobalExceptionHandler con el HTTP status correcto.
- Se manejan tambien las excepciones de Spring: MethodArgumentNotValidException (400 con detalle de campos invalidos), HttpMessageNotReadableException (400 cuando el JSON esta malformado) y AccessDeniedException (403).
- Las excepciones personalizadas son clases que extienden RuntimeException, tienen un constructor que recibe el mensaje descriptivo y estan organizadas en el paquete exception/. No usar Exception generica ni IllegalArgumentException directamente en los servicios.
- El path del ErrorResponse se obtiene inyectando HttpServletRequest en cada metodo handler para capturar el endpoint exacto que genero el error.

13.3 Transacciones (@Transactional)

- @Transactional se aplica en la capa de servicio, nunca en controllers ni repositories. Garantiza que si cualquier operacion dentro del metodo falla, todo se revierte (rollback atomico).
- Operaciones criticas que DEBEN ser transaccionales: registrar una puja y actualizar precio_actual de la subasta, cerrar una subasta y crear la orden del ganador, confirmar un pago y cambiar estados de orden/subasta/publicacion simultaneamente.
- Para metodos de solo lectura usar @Transactional(readOnly = true). Mejora el rendimiento porque Hibernate no activa el dirty checking ni genera snapshots innecesarios.
- Los metodos @Async y los listeners de eventos NO heredan la transaccion del metodo que los llama. Por eso el cierre de subastas usa @TransactionalEventListener(phase = AFTER_COMMIT) para garantizar que el evento se procesa solo despues de que la transaccion principal se haya confirmado exitosamente en la BD.
- El bloqueo optimista (@Version en la entidad Subasta) trabaja dentro de la transaccion. Si dos pujas concurrentes intentan actualizar precio_actual al mismo tiempo, la segunda lanza OptimisticLockException, que el GlobalExceptionHandler captura y retorna como 409 Conflict con mensaje claro al cliente.

13.4 Arquitectura en Capas y Responsabilidades

- Controller: recibe el request HTTP, valida con @Valid, llama al servicio y retorna ResponseEntity. Maximo 5-10 lineas por metodo. Cero logica de negocio.
- Service (interface + impl): contiene toda la logica de negocio. Orquesta repositories, lanza eventos, llama a otros servicios. Cada servicio tiene una

responsabilidad unica — UserService solo gestiona usuarios, SubastaService solo gestiona subastas.

- Repository: extiende JpaRepository. Solo contiene queries (@Query con JPQL o metodos derivados). Sin logica de negocio ni transformaciones.
- La inyeccion de dependencias siempre es por constructor (no por campo con @Autowired directo). Facilita el testing con Mockito y hace explicitas las dependencias de cada clase.
- Nunca instanciar beans de Spring con new. Siempre inyectar via constructor o usar el contexto de Spring.

13.5 Validaciones en Dos Niveles

Nivel 1 — Bean Validation en DTOs (anotaciones):

- @NotNull, @NotBlank — campos obligatorios. @Email — formato de email valido. @Size(min, max) — longitud de strings. @Min, @Max — rangos numericos (ej: calificacion entre 1 y 5). @Pattern — formato especifico (ej: DNI de 8 digitos numericos). @Positive — montos de puja mayores a cero.
- El controller usa @Valid en el @RequestBody para activar Bean Validation. Si falla, Spring lanza MethodArgumentNotValidException que el GlobalExceptionHandler convierte en 400 con la lista de campos invalidos.

Nivel 2 — Validacion de logica de negocio en servicios:

- Reglas que no pueden expresarse con anotaciones: verificar que el pujador no es el vendedor, que el monto supera el precio actual, que la subasta sigue activa, que el DNI no esta duplicado. Se lanzan excepciones personalizadas desde el servicio.
- A nivel de base de datos: constraints de unicidad (@Column(unique=true) para email y DNI), @NotNull en columnas obligatorias, y check constraints en enums via @Enumerated(EnumType.STRING).

13.6 Seguridad y Autenticacion

- El JWT secret, credenciales de BD y configuracion de mail se cargan exclusivamente desde variables de entorno. Nunca hardcodear valores sensibles en el codigo ni en application.properties.
- El JwtAuthenticationFilter se ejecuta en cada request protegido: extrae el token del header Authorization, lo valida con JwtService y carga el usuario en el SecurityContext. Si el token es invalido o expirado, retorna 401 sin llegar al controller.
- Para obtener el usuario autenticado en un servicio se usa SecurityContextHolder.getContext().getAuthentication() — nunca recibir el userId como parametro del request porque puede ser manipulado por el cliente.
- @PreAuthorize("hasRole('ADMIN')") protege endpoints administrativos a nivel de metodo. Complementa la configuracion de rutas en SecurityConfig y añade una segunda capa de verificacion.
- El archivo .env y application-local.properties deben estar en .gitignore. Nunca subir credenciales reales al repositorio. En su lugar, documentar las variables requeridas en el README con valores de ejemplo.

13.7 Testing

- Nomenclatura BDD obligatoria en todos los tests: shouldXxxWhenYyy. Ejemplo: shouldThrowInvalidBidExceptionWhenMontoMenorAlActual, shouldReturn201WhenSubastaCreatedSuccessfully.
- Tests de repositorio con @DataJpaTest + TestContainers (PostgreSQL real en Docker). No usar H2 en memoria porque el comportamiento difiere de PostgreSQL en constraints y tipos de datos.
- Tests de servicio con Mockito: mockear todos los repositorios y servicios externos (EmailService, StripeService). Solo probar la logica del metodo bajo test, no sus dependencias.
- Tests de controller con @WebMvcTest + MockMvc: verificar status codes HTTP, estructura del JSON de respuesta, headers y manejo de errores. Mockear el servicio con @MockBean.
- Estructura Given / When / Then dentro de cada test para que sea legible: Given (preparar datos y mocks), When (ejecutar el metodo), Then (verificar resultado con assertThat o verify).

13.8 Convenciones deCodigo

- Todos los nombres (clases, metodos, variables, endpoints) en ingles. Sin abreviaturas crípticas: usar subasta en vez de sub, publicacion en vez de pub.
- Metodos cortos y cohesivos: maximo 20-30 lineas. Si un metodo crece mas, extraer logica a metodos privados con nombres descriptivos.
- Sin comentarios que expliquen el que (el codigo debe ser autoexplicativo). Comentarios solo para el por que cuando la decision no es obvia (ej: por que se usa LAZY en una relacion especifica).
- Commits frecuentes y descriptivos siguiendo el formato: feat: agregar endpoint de pujas, fix: corregir validacion de monto minimo, test: agregar tests de SubastaService. Ramas por feature (feature/auction-bidding) y pull requests con code review antes de merge a main.
- Paginacion obligatoria en todos los endpoints de listado (GET /publicaciones, GET /subastas, GET /notificaciones). Usar Pageable de Spring Data con parametros page y size en el query param.
- Usar enums de Java para todos los estados (SubastaEstado, OrdenEstado, PagoEstado, UserRole) y almacenarlos como STRING en la BD con @Enumerated(EnumType.STRING). Facilita la legibilidad de los datos en la BD y evita errores por cambios de orden.

Pestaña 3

Aquí van todas las buenas prácticas:

13. Buenas Prácticas de Desarrollo

Esta sección describe los patrones, convenciones y decisiones técnicas que el equipo debe aplicar de forma consistente en todo el código del proyecto Yala.

13.1 DTOs (Data Transfer Objects)

Nunca exponer una entidad JPA directamente en un endpoint. Siempre usar DTOs especializados según la operación:

- **XxxRequest** — datos que llegan del cliente (POST/PUT). Incluyen anotaciones de validación (`@NotNull`, `@Email`, `@Size`, `@Min`).
 - **XxxResponse** — datos que se retornan al cliente. Solo los campos necesarios, nunca contraseñas ni datos sensibles como el DNI completo o el CCI.
 - **XxxDetailResponse** — versión extendida del response para vistas de detalle. Por ejemplo, `AuctionDetailResponse` incluye la lista de pujas, el vendedor y la publicación completa.
 - El mapeo entidad → DTO se centraliza en métodos estáticos dentro del propio DTO (`from(Entidad e)`) o usando `ModelMapper` configurado como bean. No dispersar la lógica de mapeo en los servicios.
 - Los DTOs usan Lombok (`@Getter`, `@Setter`, `@Builder`, `@NoArgsConstructor`, `@AllArgsConstructor`) para evitar código boilerplate.
 - `ErrorResponse` es un DTO especial retornado por el `GlobalExceptionHandler`. Siempre incluye: `timestamp`, `status`, `error`, `message` y `path`.
-

13.2 Manejo Global de Excepciones (@RestControllerAdvice)

- Existe una única clase `GlobalExceptionHandler` anotada con `@RestControllerAdvice`. Ningún controller ni servicio atrapa excepciones con try/catch para construir respuestas HTTP — eso es responsabilidad exclusiva del handler global.
- Cada excepción personalizada del proyecto (`ResourceNotFoundException`, `InvalidBidException`, etc.) tiene su propio método `@ExceptionHandler` con el HTTP status correcto.

- Se manejan también las excepciones de Spring: `MethodArgumentNotValidException` (400 con detalle de campos inválidos), `HttpMessageNotReadableException` (400 cuando el JSON está malformado) y `AccessDeniedException` (403).
 - Las excepciones personalizadas extienden `RuntimeException`, tienen un constructor que recibe el mensaje descriptivo y están organizadas en el paquete `exception/`. No usar `Exception` genérica ni `IllegalArgumentException` directamente en los servicios.
 - El `path` del `ErrorResponse` se obtiene inyectando `HttpServletRequest` en cada método handler para capturar el endpoint exacto que generó el error.
-

13.3 Transacciones (@Transactional)

- `@Transactional` se aplica en la capa de servicio, nunca en controllers ni repositories. Garantiza que si cualquier operación dentro del método falla, todo se revierte (rollback atómico).
 - Operaciones críticas que DEBEN ser transaccionales: registrar una puja y actualizar `precio_actual` de la subasta, cerrar una subasta y crear la orden del ganador, confirmar un pago y cambiar estados de orden/subasta/publicación simultáneamente.
 - Para métodos de solo lectura usar `@Transactional(readOnly = true)`. Mejora el rendimiento porque Hibernate no activa el dirty checking ni genera snapshots innecesarios.
 - Los métodos `@Async` y los listeners de eventos NO heredan la transacción del método que los llama. Por eso el cierre de subastas usa `@TransactionalEventListener(phase = AFTER_COMMIT)` para garantizar que el evento se procesa solo después de que la transacción principal se haya confirmado en la BD.
 - El bloqueo optimista (`@Version` en la entidad `Subasta`) trabaja dentro de la transacción. Si dos pujas concurrentes intentan actualizar `precio_actual` al mismo tiempo, la segunda lanza `OptimisticLockException`, que el `GlobalExceptionHandler` captura y retorna como 409 Conflict.
-

13.4 Arquitectura en Capas y Responsabilidades

- **Controller** — recibe el request HTTP, valida con `@Valid`, llama al servicio y retorna `ResponseEntity`. Máximo 5-10 líneas por método. Cero lógica de negocio.
- **Service (interface + impl)** — contiene toda la lógica de negocio. Orquesta repositorios, lanza eventos, llama a otros servicios. Cada servicio tiene una

responsabilidad única: `UserService` solo gestiona usuarios, `AuctionService` solo gestiona subastas.

- **Repository** — extiende `JpaRepository`. Solo contiene queries (`@Query` con JPQL o métodos derivados). Sin lógica de negocio ni transformaciones.
 - La inyección de dependencias se realiza **exclusivamente** mediante constructor con campos declarados como `private final`. No usar `@Autowired` en campos ni en setters. `@RequiredArgsConstructor` de Lombok genera el constructor automáticamente a partir de los campos `final`, eliminando código repetitivo.
 - Nunca instanciar beans de Spring con `new`. Siempre inyectar vía constructor.
-

13.5 Validaciones en Dos Niveles

Nivel 1 — Bean Validation en DTOs:

- `@NotNull`, `@NotBlank` — campos obligatorios.
- `@Email` — formato de email válido.
- `@Size(min, max)` — longitud de strings.
- `@Min`, `@Max` — rangos numéricos (ej: calificación entre 1 y 5).
- `@Pattern` — formato específico (ej: DNI de exactamente 8 dígitos numéricos, nombres solo con letras y espacios).
- `@Positive` — montos de puja mayores a cero.
- El controller usa `@Valid` en el `@RequestBody` para activarlas. Si falla, Spring lanza `MethodArgumentNotValidException` que el `GlobalExceptionHandler` convierte en 400 con la lista de campos inválidos.

Nivel 2 — Validación de lógica de negocio en servicios:

- Reglas que no pueden expresarse con anotaciones: verificar que el pujador no es el vendedor, que el monto supera el precio actual, que la subasta sigue activa, que el DNI no está duplicado. Se lanzan excepciones personalizadas desde el servicio.
 - A nivel de base de datos: constraints de unicidad (`@Column(unique=true)` para email y DNI), `@NotNull` en columnas obligatorias, enums almacenados como `@Enumerated(EnumType.STRING)`.
-

13.6 Seguridad y Autenticación

- El JWT secret y las credenciales de BD se cargan exclusivamente desde variables de entorno definidas en un archivo `.env`. Nunca hardcodear valores

sensibles en el código ni en `application.properties`. El `application.properties` referencia las variables usando la sintaxis `${VARIABLE_NAME}`.

- El `.env` debe estar en `.gitignore`. Se incluye en el repositorio únicamente un `.env.example` con la lista de variables sin valores reales, y el `application.properties` con placeholders.
 - El `JwtAuthenticationFilter` se ejecuta en cada request protegido: extrae el token del header `Authorization`, lo valida con `JwtService` y carga el usuario en el `SecurityContext`. Si el token es inválido o expirado, retorna 401 sin llegar al controller.
 - Para obtener el usuario autenticado en un servicio se usa `SecurityContextHolder.getContext().getAuthentication()` — nunca recibir el `userId` como parámetro del request porque puede ser manipulado por el cliente.
 - `@PreAuthorize("hasRole('ADMIN')")` protege endpoints administrativos a nivel de método, añadiendo una segunda capa de verificación sobre la configuración de rutas en `SecurityConfig`.
-

13.7 Testing

- Nomenclatura BDD obligatoria en todos los tests: `shouldXxxWhenYyy`. Ejemplos:
`shouldThrowInvalidBidExceptionWhenMontoMenorAlActual,`
`shouldReturn201WhenAuctionCreatedSuccessfully.`
 - Tests de repositorio con `@DataJpaTest` + `TestContainers` (PostgreSQL real en Docker). No usar H2 en memoria porque el comportamiento difiere en constraints y tipos de datos.
 - Tests de servicio con Mockito: mockear todos los repositorios y servicios externos (`EmailService`, `StripeService`). Solo probar la lógica del método bajo test. Gracias a la inyección por constructor con `private final`, Mockito puede inyectar mocks directamente sin levantar el contexto de Spring.
 - Tests de controller con `@WebMvcTest` + `MockMvc`: verificar status codes HTTP, estructura del JSON de respuesta, headers y manejo de errores. Mockear el servicio con `@MockBean`.
 - Estructura **Given / When / Then** dentro de cada test: Given (preparar datos y mocks), When (ejecutar el método), Then (verificar resultado con `assertThat` o `verify`).
-

13.8 Convenciones de Código

- Todos los nombres (clases, métodos, variables, endpoints) en inglés. Sin abreviaturas crípticas: `auction` en vez de `auct`, `publication` en vez de `pub`.
- Métodos cortos y cohesivos: máximo 20-30 líneas. Si un método crece más, extraer lógica a métodos privados con nombres descriptivos.
- Sin comentarios que expliquen el *qué* (el código debe ser autoexplicativo). Comentarios solo para el *por qué* cuando la decisión no es obvia.
- **Conventional Commits** obligatorio. Formato: `tipo(scope): descripcion corta`. Tipos permitidos: `feat` (nueva funcionalidad), `fix` (corrección de bug), `test` (agregar o corregir tests), `refactor` (mejora sin cambio funcional), `docs` (documentación), `chore` (configuración o dependencias). Ejemplos: `feat(auction): agregar endpoint de pujas`, `fix(bid): corregir validación de monto mínimo`, `test(user): agregar tests de UserService`. Ramas por feature con el patrón `feature/nombre-funcionalidad` y pull requests con code review obligatorio antes de merge a `main`.
- Paginación obligatoria en todos los endpoints de listado usando `Pageable` con parámetros `page` y `size` como query params.
- Usar enums de Java para todos los estados y categorías: `SubastaEstado`, `OrdenEstado`, `PagoEstado`, `UserRole`, `CategoriaItem` (POKEMON_TCG, FUNKO_POP, COMIC, OTRO), `CondicionItem` (PSA_10, PSA_9, PSA_8, PSA_7_INFERIOR, SIN_GRADUAR). Almacenar como `STRING` en la BD con `@Enumerated(EnumType.STRING)`.

13.9 Estructura de Carpetas por Entidad

El proyecto se organiza por entidad de dominio, no por capa técnica. Cada entidad tiene su propia carpeta dentro de `com.yala`, y dentro de ella cada componente tiene su subcarpeta:

- Estructura base: `com.yala.auction/` contiene `controller/`, `service/`, `repository/`, `model/` y `dto/`. El mismo patrón aplica para `user/`, `bid/`, `order/`, `payment/`, `review/`, `notification/`.
- La carpeta `dto/` dentro de cada entidad contiene todos sus request y response DTOs. Ejemplo: `com.yala.auction/dto/AuctionCreateRequest.java`, `AuctionResponse.java`, `AuctionDetailResponse.java`.
- La carpeta `service/` contiene la interfaz (`AuctionService.java`) y su implementación (`AuctionServiceImpl.java`) en el mismo nivel. No crear subcarpetas `interfaces/` e `impl/` separadas.
- Componentes transversales al mismo nivel: `com.yala.security/` (JWT, filtros, `UserDetailsService`), `com.yala.config/` (`SecurityConfig`,

AsyncConfig, ModelMapperConfig), `com.yala.exception/` (excepciones personalizadas + GlobalExceptionHandler), `com.yala.event/` (clases de eventos de dominio).

13.10 Estructura de Endpoints y Path Parameters

- Todos los endpoints siguen la estructura `/api/v1/{recurso-en-plural}`. El prefijo `/api/v1` se define una sola vez en `@RequestMapping` a nivel de clase en el controller, nunca repetido en cada método.
 - Usar `@PathVariable` para identificadores de recursos en la URL: `GET /api/v1/auctions/{auctionId}`, `POST /api/v1/auctions/{auctionId}/bids`, `GET /api/v1/users/{userId}/reviews`. El nombre de la variable en la anotación debe coincidir exactamente con el nombre del parámetro del método.
 - Usar `@RequestParam` para filtros opcionales y paginación: `GET /api/v1/auctions?category=POKEMON_TCG&page=0&size=10`. No poner filtros como path variables porque hacen la URL rígida.
 - Verbos HTTP correctos: `GET` para consultas (nunca modifica datos), `POST` para crear (retorna 201), `PUT` para reemplazar completo, `PATCH` para actualizaciones parciales, `DELETE` para eliminar (retorna 204).
 - Recursos anidados para relaciones naturales de dominio: `POST /api/v1/auctions/{auctionId}/bids` en lugar de `POST /api/v1/bids` con `auctionId` en el body. Deja claro en la URL que una puja pertenece a una subasta específica.
-

13.11 Dependencias Maven del Proyecto

No agregar dependencias sin justificación técnica clara. Las dependencias del proyecto son:

- `spring-boot-starter-data-jpa` — ORM con Hibernate para entidades y repositorios con PostgreSQL.
- `spring-boot-starter-security` — autenticación, autorización y filtros de seguridad.
- `spring-boot-starter-web` — controladores REST, Jackson y Tomcat embebido.
- `spring-boot-starter-validation` — Bean Validation con Hibernate Validator para anotaciones en DTOs.
- `spring-boot-starter-mail` — envío de emails con `JavaMailSender` para confirmaciones de registro, subasta ganada y estado de pago.

- **modelmapper 3.2.1** — mapeo automático entidad → DTO, configurado en modo estricto como bean.
- **jjwt-api + jjwt-impl + jjwt-jackson 0.12.6** — generación, firma y validación de tokens JWT.
- **postgresql (runtime)** — driver JDBC para la conexión con PostgreSQL.
- **lombok** — elimina boilerplate con **@Getter**, **@Setter**, **@Builder**, **@NoArgsConstructor**, **@AllArgsConstructor** y **@RequiredArgsConstructor** (este último clave para la inyección por constructor con **private final**).
- **Scope test:** **spring-boot-starter-test** (JUnit 5 + Mockito + AssertJ), **spring-security-test** (**@WithMockUser** en tests de controller), TestContainers con módulo PostgreSQL para tests de integración con base de datos real.

<dependencies>

```
<dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-data-jpa</artifactId>

</dependency>

<dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-security</artifactId>

</dependency>

<dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-webmvc</artifactId>

</dependency>

<dependency>

    <groupId>org.modelmapper</groupId>

    <artifactId>modelmapper</artifactId>

    <version>3.2.1</version>
```

```
</dependency>

<dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-validation</artifactId>

</dependency>

<dependency>

    <groupId>io.jsonwebtoken</groupId>

    <artifactId>jjwt-api</artifactId>

    <version>0.12.6</version>

</dependency>

<dependency>

    <groupId>io.jsonwebtoken</groupId>

    <artifactId>jjwt-impl</artifactId>

    <version>0.12.6</version>

    <scope>runtime</scope>

</dependency>

<dependency>

    <groupId>io.jsonwebtoken</groupId>

    <artifactId>jjwt-jackson</artifactId>

    <version>0.12.6</version>

    <scope>runtime</scope>

</dependency>

<dependency>

    <groupId>org.postgresql</groupId>

    <artifactId>postgresql</artifactId>

    <scope>runtime</scope>

</dependency>

<dependency>
```

```
<groupId>org.projectlombok</groupId>

<artifactId>lombok</artifactId>

<optional>true</optional>

</dependency>

<dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-data-jpa-test</artifactId>

    <scope>test</scope>

</dependency>

<dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-security-test</artifactId>

    <scope>test</scope>

</dependency>

<dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-webmvc-test</artifactId>

    <scope>test</scope>

</dependency>

</dependencies>
```